

Comparative Study of Logistic Regression Implementations for Breast Cancer Detection

Siddh Sharma

Roll Number: 25135150

Mechanical Engineering

Indian Institute of Technology (BHU), Varanasi

1 Introduction

The aim of the report is to construct a Logistic Regression model to classify breast cancer tumors as malignant or benign based on the features provided in the dataset. The report includes three implementations and their comparison. The implementations are: Pure Python (from scratch), NumPy (vectorized operations and matrix multiplication), and Scikit-Learn. The report showcases the mathematical reasoning behind Logistic Regression and Gradient Descent. The models were evaluated based on the metrics: Accuracy, Precision, Recall, F1 Score, and ROC-AUC. The report also compares the speed of convergence of the implementations involved and its reasoning.

The prediction in Logistic Regression is computed using the sigmoid function:

$$\hat{y} = \sigma(Xw + b) = \frac{1}{1 + e^{-(Xw+b)}}$$

2 Dataset Description

The Breast Cancer Wisconsin (Diagnostic) Dataset is the dataset used for this report. It consists of 568 instances, 30 numerical features, and 1 target variable (diagnosis). The target variable consists of two categories: M (Malignant) = 1 and B (Benign) = 0. There are no missing values and hence there is no need for imputation. The dataset consists of 10 measurements for each cell nucleus: radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension. Each measurement has 3 statistics: mean, standard error (SE), and worst, thus making up the 30 numerical features.

3 Data Preprocessing

The dataset has no missing data therefore no imputation is required. The diagnosis column was encoded (M=1, B=0) to prepare it for Logistic Regression as the model requires numerical targets. The dataset was given an 80/20 train-validation split (with `random_state=42` for reproducibility). 80% is enough for the model to train on and 20% is sufficient for reliable validation given the sample consists of 568 instances.

Standard Scaling was applied as gradient descent works well when the data is on a uniform scale. Since the dataset consists of features which have a lot of difference in their ranges, without applying Standard Scaling the time of convergence would increase. Standard scaling uses the formula:

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

Both the training and validation sets use the train mean and standard deviation for scaling to avoid data leakage.

4 Implementation

4.1 Pure Python Implementation

The implementation involves making Logistic Regression completely from scratch using only core Python with no external libraries. The prediction probabilities were computed by applying the sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

to the weighted sum. The model involves Gradient Descent as the optimization technique and Binary Cross Entropy as the loss function, as it is a convex function and guarantees convergence to the global minimum:

$$J = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

The gradients were computed using nested loops and lists:

$$\frac{\partial J}{\partial w_j} = \frac{1}{n} \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij}$$

The weights and bias were initialized to zero across all implementations to maintain uniformity for fair comparison. The learning rate was set to 0.01 and the convergence threshold to 10^{-5} . The weights and bias were updated according to:

$$w := w - \eta \nabla J(w)$$

4.2 NumPy Implementation

The implementation has the same logic as the Pure Python implementation but replaces nested loops with vectorized operations and matrix multiplications using NumPy. Predictions were computed using:

$$\hat{y} = \sigma(Xw + b)$$

The gradients were computed using matrix operations:

$$\nabla w = \frac{1}{n} X^T (\hat{y} - y)$$

The weights and bias were updated according to:

$$w := w - \eta \nabla J(w)$$

The hyperparameters were the same as the Pure Python implementation to ensure a fair comparison.

4.3 Scikit-Learn Implementation

In this implementation, the `LogisticRegression` class from Scikit-Learn was used to train the model on the training set and make predictions on the validation set. It does not use gradient descent; rather it uses the LBFGS solver which is optimized to be far more efficient than the gradient descent method used in the above implementations.

4.4 Metrics used for Evaluation

- **Accuracy:** Measures overall correctness of the model.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision:** Measures how many predicted malignant cases were actually malignant.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall:** Measures how many actual malignant cases were correctly identified.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1 Score:** Harmonic mean of Precision and Recall.

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **Confusion Matrix:** A 2×2 matrix summarizing TP, TN, FP, and FN.

$$\begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix}$$

- **ROC-AUC:** The ROC curve plots Recall on the y-axis against the False Positive Rate on the x-axis. The AUC (area under the curve) summarizes model performance — the higher the better.

4.5 Threshold

Threshold is one of the most important hyperparameters in Logistic Regression. It classifies an instance based on whether its predicted probability falls above or below the threshold. All the probabilities above the threshold are classified as one class and below as the other. This controls the precision-recall tradeoff.

In this report, the threshold was set to 0.45 across all implementations. Since this is a cancer detection problem, a lower threshold was chosen to improve recall. Classifying a malignant case as benign (false negative) is far more dangerous than classifying a benign case as malignant (false positive), as the person in the latter case can take further tests to clarify the result.

5 Results and Analysis

Metric	Pure Python	NumPy	Scikit-Learn
Accuracy	0.9825	0.9825	0.9825
Precision	0.9583	0.9583	1.0000
Recall	1.0000	1.0000	0.9565
F1 Score	0.9787	0.9787	0.9778
ROC-AUC	0.9994	0.9994	0.9939
Convergence Time	13.65s	4.32s	0.02s

Table 1: Comparison of Different Implementations

Insights:

- Accuracy is identical across all three implementations (98.25%), confirming that the Pure Python and NumPy scratch implementations are mathematically correct and produce results comparable to a highly optimized library.
- The Pure Python and NumPy implementations with the same hyperparameters produce identical results across all metrics. This is expected as their mathematical background is the same — the only difference was the method of computation.

- Scikit-Learn has higher precision (100%) but lower recall (95.65%), while Pure Python and NumPy have higher recall (100%) but lower precision (95.83%) at a threshold of 0.45. For cancer detection, recall is the more critical metric since missing a malignant case is far more dangerous than a false alarm.
- ROC-AUC of Pure Python and NumPy (0.9994) outperforms that of Scikit-Learn (0.9939), showing that gradient descent with a threshold of 0.45 achieves better classification in probability space.
- NumPy is 3.16 times faster than Pure Python due to vectorized operations. Sklearn is 682 times faster than Pure Python due to its highly optimized LBFGS solver which converges in fewer steps than vanilla gradient descent.

6 Visualizations

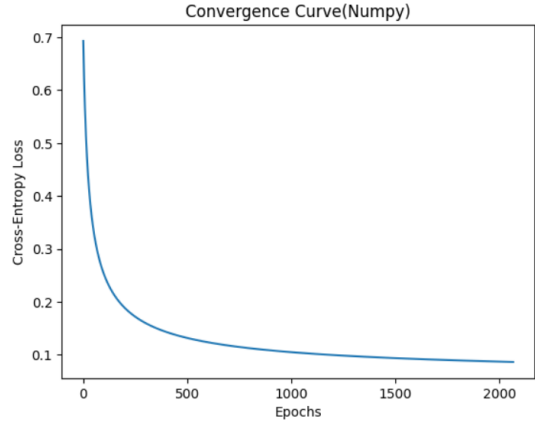
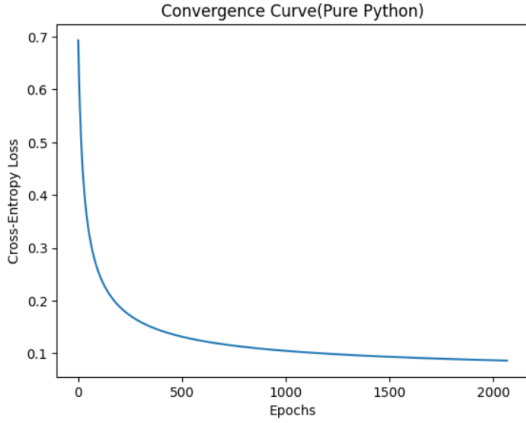


Figure 1: Loss vs Epoch Curve (Pure Python)

Figure 2: Loss vs Epoch Curve (NumPy)

The loss first decreased steeply in the first 200 epochs and then started to flatten, showing that most of the learning happens early. Convergence takes place at epoch 2067 in both curves when the convergence criterion was 10^{-5} and learning rate was 0.01. The graphs of both curves are identical, confirming their mathematical equivalence.

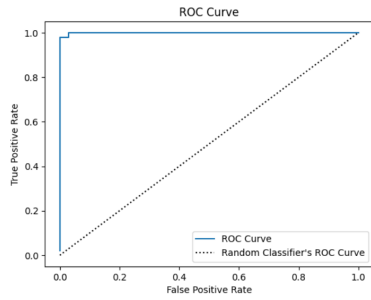


Figure 3: ROC Curve (Pure Python)

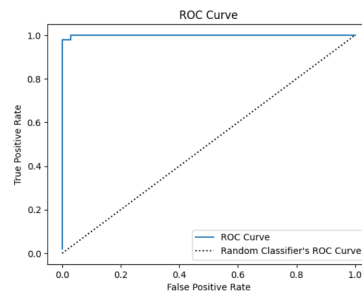


Figure 4: ROC Curve (NumPy)

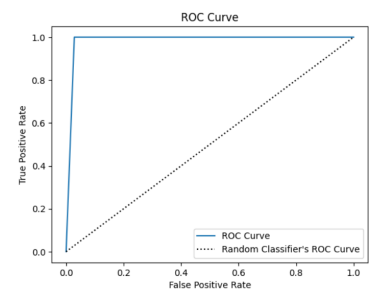


Figure 5: ROC Curve (Scikit-Learn)

All three implementations produce curves close to the top-left corner with AUC above 0.99, indicating near-perfect classification.

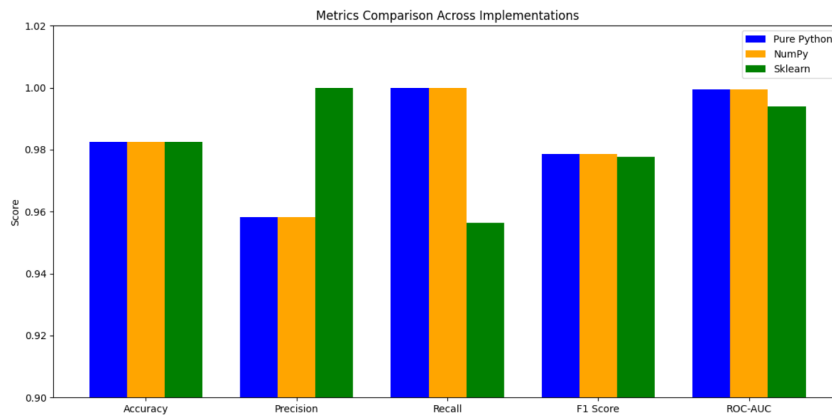


Figure 6: Metrics Comparison Across Implementations

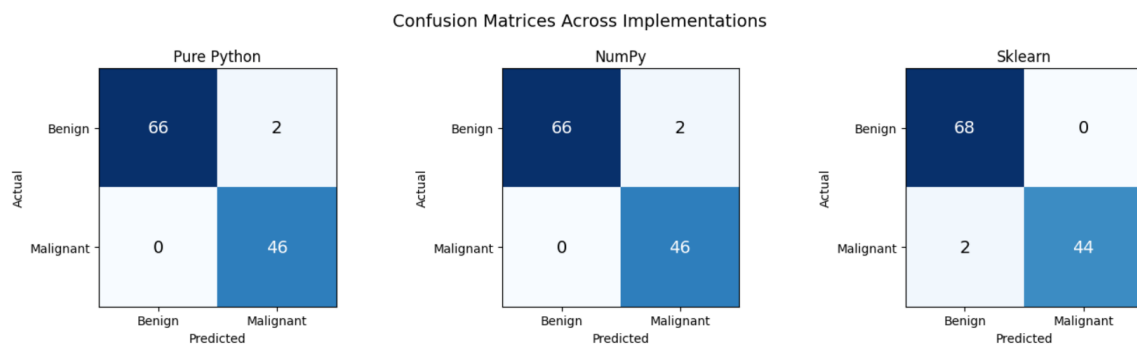


Figure 7: Confusion Matrices — All Three Implementations

Pure Python and NumPy have 0 false negatives (no missed malignant cases) but 2 false

positives. Sklearn has 0 false positives but 2 false negatives. For cancer detection, 0 false negatives is the preferred outcome.

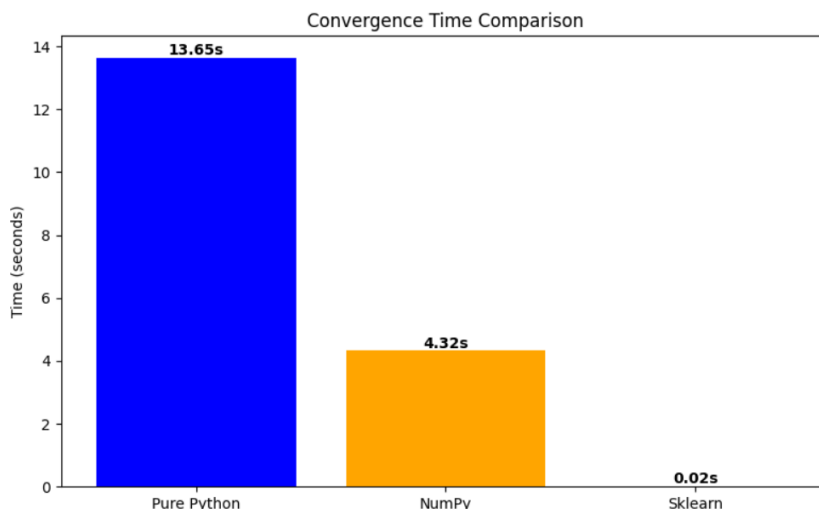


Figure 8: Convergence Time Comparison

NumPy is 3.16 times faster than Pure Python, whereas Scikit-Learn is 682 times faster than Pure Python.

7 Conclusion

All three implementations achieved 98.25% accuracy, showing that scratch implementations in Python and NumPy can achieve comparable results to highly optimized models and proving their mathematical correctness. Pure Python and NumPy achieved identical results confirming that vectorized operations and nested loops produce the same results — the only difference being convergence speed, in which NumPy dominates due to parallelized operations. Sklearn was also much faster than the Pure Python implementation due to its LBFGS optimizer. Gradient Descent implementations (Part 1 and 2) achieved 100% recall — no malignant case was missed — which is the most important outcome for cancer detection. Sklearn achieved 100% precision but missed 2 malignant cases, which is not a desirable outcome for medical diagnosis. This project provided a deep understanding of how gradient descent works internally, how logistic regression is optimized, and the role of hyperparameters such as threshold and learning rate in classification problems.